

КТ-3.1 — Руководство пользователя

User Guide

Кадочников Лев Петрович (гр. МФТИ-1-2024)

Команда: «Черепашки Ninja»

2026-06-13

1. ЧАСТЬ 1: Руководство администратора

1.1. Системные требования

Компонент	Минимум	Рекомендуется
ОС	Windows 10 / Ubuntu 20.04	Windows 11 / Ubuntu 24.04
Компилятор	Clang 22 (или clang-cl 22 для Windows)	—
CMake	3.30+	—
Бэкенд сборки	Ninja 1.13+	—
Vulkan SDK	1.4.350+	—
GPU	Поддержка Vulkan 1.4	Дискретная GPU с 4+ GB VRAM
ОЗУ	8 GB	16 GB
Диск	5 GB (исходники + build + assets)	10 GB SSD

Сетевые порты: не требуются (приложение однопроцессное, не открывает портов).

1.2. Инструкция по развёртыванию (Linux)

```
# 1. Клонирование репозитория с подмодулями
git clone --recursive https://github.com/Leeleit/ProjectV.git
cd ProjectV

# 2. Конфигурация CMake (preset linux-clang-debug)
cmake --preset linux-clang-debug
#   → создаст build/linux-clang-debug/ с Ninja-конфигурацией

# 3. Сборка (~ 5-15 минут на 16 ядрах)
cmake --build build/linux-clang-debug --parallel

# 4. Запуск runtime-смоук-теста
./tools/linux/Invoke-ProjectVRuntimeSmoke.sh
#   → генерирует 6 captures (FINAL, SHDW, CSM, CTSH, AOCC, LOCL)
#   → в build/linux-clang-debug/lookdev-captures/

# 5. Запуск приложения напрямую
./build/linux-clang-debug/src/app/ProjectV
```

1.3. Инструкция по развёртыванию (Windows)

```
# 1. Клонирование
git clone --recursive https://github.com/Leeleit/ProjectV.git
cd ProjectV

# 2. Конфигурация (preset windows-clang-debug)
cmake --preset windows-clang-debug

# 3. Сборка (clang-cl + Ninja)
cmake --build build/windows-clang-debug --parallel

# 4. Запуск runtime-смоук-теста
```

```
powershell -ExecutionPolicy Bypass -File tools\windows\Invoke-ProjectVRuntimeSmoke.ps1
# → генерирует 6 captures (FINAL, SHDW, CSM, CTSH, AOCC, LOCL)
# → в build\windows-clang-debug\lookdev-captures\
```

```
# 5. Запуск приложения напрямую
build\windows-clang-debug\src\app\ProjectV.exe
```

1.4. Переменные окружения

Имя	Назначение	Значение по умолчанию
VULKAN_SDK	Путь к Vulkan SDK (для шейдеров и валидации)	автоопределение CMake
PROJECTV_ENABLE_VALIDATION	Включить Vulkan validation layers	ON (debug), OFF (release)
PROJECTV_RENDERER_DEBUG	Поднять дебаг-HUD (FPS, draw calls, frame time)	OFF
PROJECTV_ASSET_DIR	Каталог с glTF-ассетами	assets/
PROJECTV_SCENE_PRESET	Пресет сцены при старте	VoxelLab
VK_INSTANCE_LAYERS	Список Vulkan-слоёв (для отладки)	VK_LAYER_KHRONOS_validation

1.5. Пресеты сцен

ProjectV поставляется с пресетами сцен, демонстрирующими разные аспекты движка:

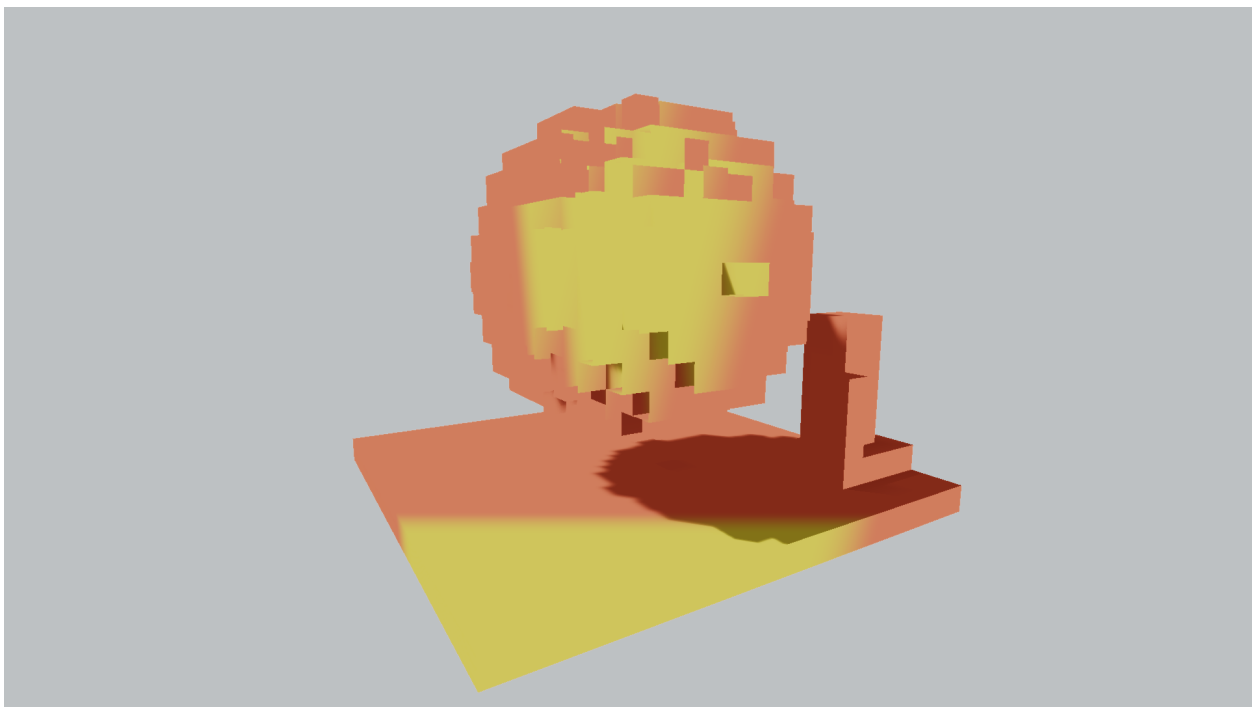
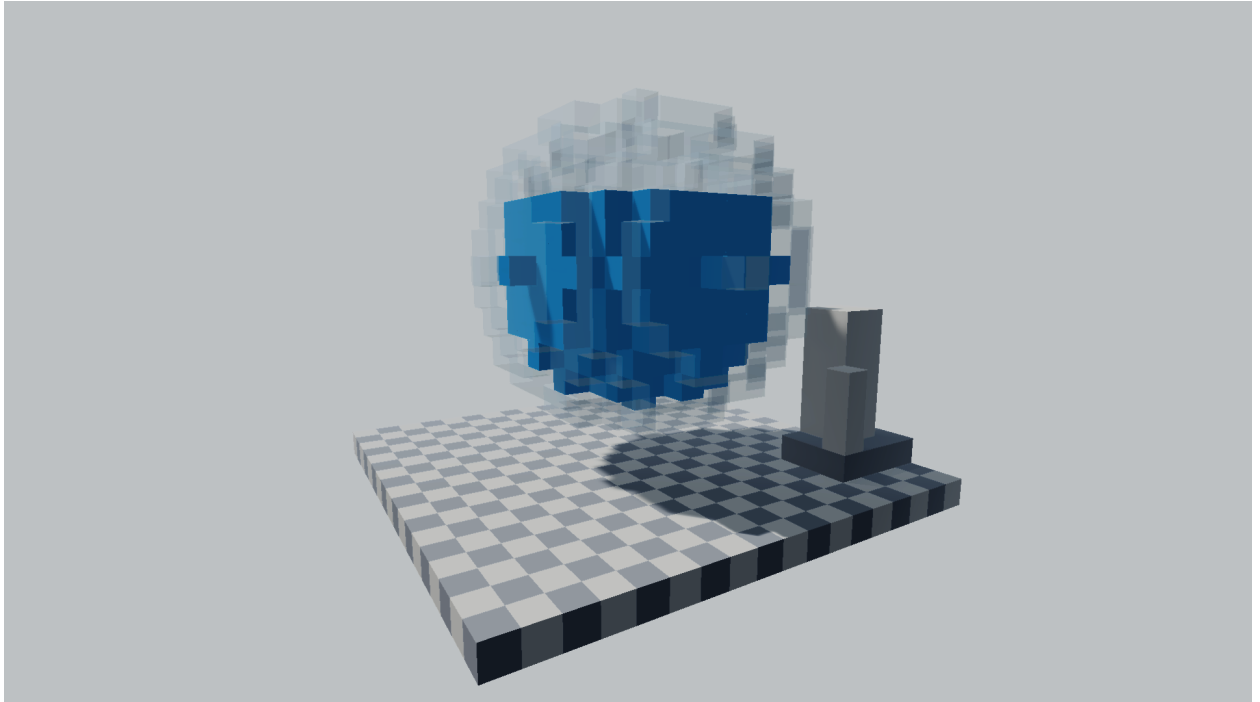
Пресет	Назначение
VoxelLab	Базовая сцена с оболочкой, opaque-anchor'ами и fluid-зоной
FlatBenchmark	Плоская платформа 64×64 без объектов — baseline для тени/мешинга
TransparencyStress	Сетка стеклянных столбов 2×2, высота 4–12 — нагрузка на alpha-blending
ChunkGrid	Вертикальные столбы на границах чанков — лес для проверки длинных теней
MeshingStress	Объём из вокселей в шахматном 3D-порядке — стресс-тест greedy-мешинга

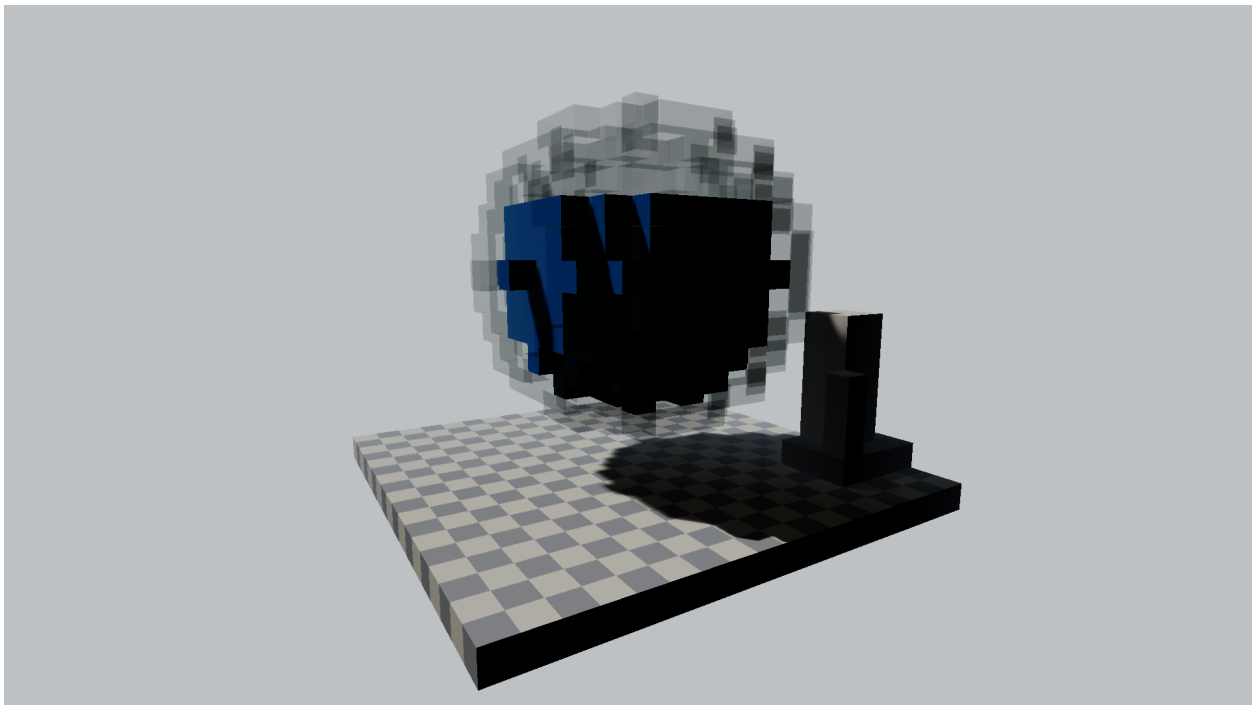
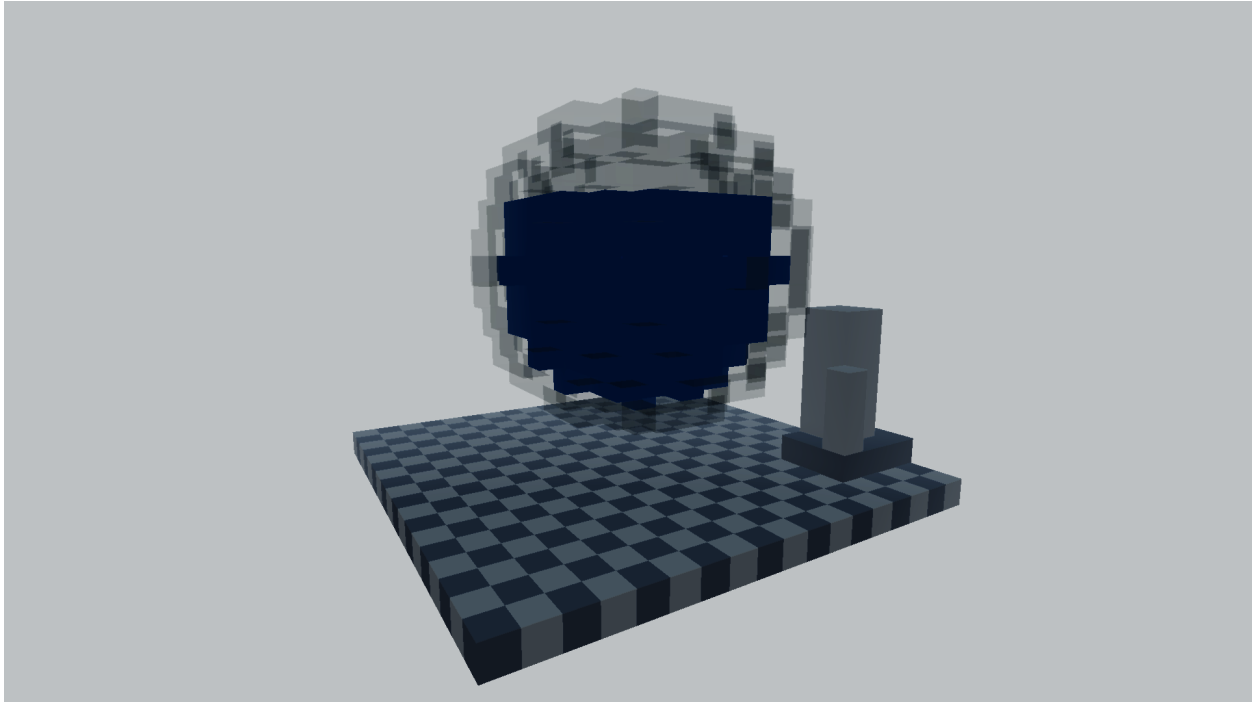
Реализация в коде (src/voxel/VoxelWorld.cpp):

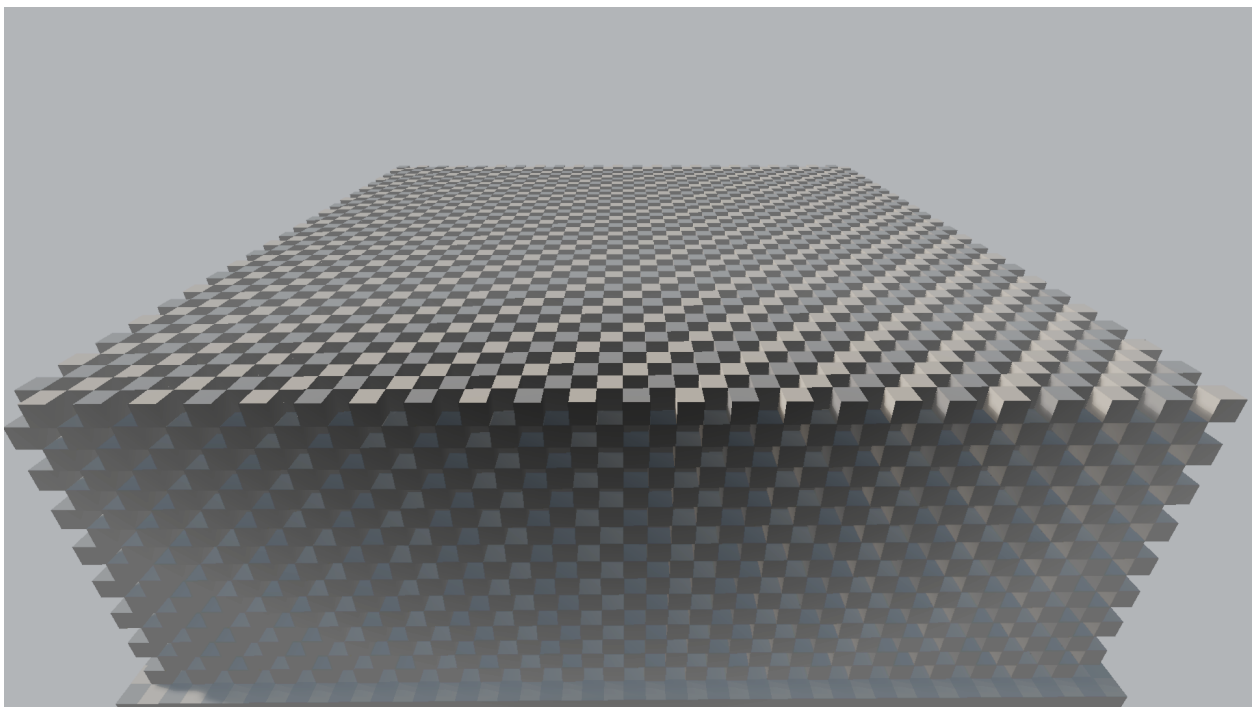
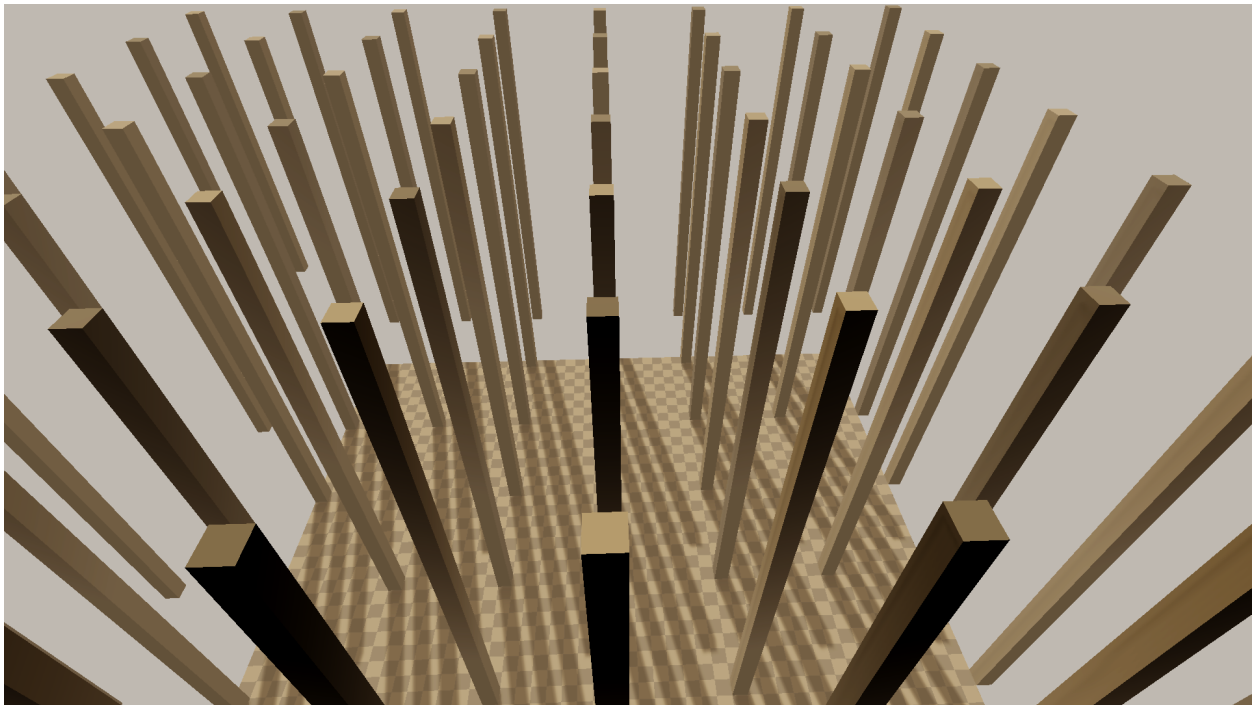
- VoxelLab — BuildVoxelLabSceneworld() → BuildVoxelLabShellAndFluid + BuildVoxelLabOpaqueAnchors.
- FlatBenchmark — BuildFlatBenchmarkSceneworld() (только CreateSceneworldWithFloor + BuildCheckerboardFloor).
- TransparencyStress — BuildTransparencyStressSceneworld() → BuildTransparencyStressColumns (стеклянные столбы в BuildTransparencyStressColumns, высоты 4–12 от (x+z)%8).
- ChunkGrid — BuildChunkGridSceneworld() → BuildChunkGridMarkers (столбы на каждой границе чанка, чередование FloorWhite/FloorGray по (chunkX+chunkZ)%2).
- MeshingStress — BuildMeshingStressSceneworld() → BuildMeshingStressVolume (объём через (x+y+z)%2, чередование FloorWhite/FloorGray по (x+z)%2, высота до 16).

Кроме того, в коде есть fluid-мешинг (voxel_fluid.comp + VoxelMaterial::Fluid=2) — он не выделен в отдельный пресет, но может быть включён через PROJECTV_SCENE_PRESET в любую сцену, если в её JSON указано использование Fluid материала.

Скриншоты (VoxelLab и стресс-пресеты):







2. ЧАСТЬ 2: Руководство пользователя

2.1. Обзор интерфейса

Графическое окно ProjectV — single-window desktop-приложение. Основные элементы:

#	Элемент	Описание
1	Главный viewport	Рендер сцены с активной камерой (WASD + мышь)
2	Debug HUD (опц.)	FPS, draw calls, frame time (вкл. G)

2.2. Управление

Полный список кнопок — в `src/app/InputActions.cpp` (`BindAction` вызовы). Ниже — основные.

Клавиша	Действие
W / S / A / D	Движение камеры (вперёд / назад / влево / вправо)
Space	Подъём камеры
LShift / RShift	Спуск камеры
LCtrl / RCtrl	Ускорение движения
LAlt / RAlt	Замедление движения
ЛКМ (mouse)	Удалить воксель под курсором (raycast)
ПКМ (mouse)	Разместить воксель под курсором (raycast)
Tab	Переключить relative mouse mode (для FPS-управления)
F1	Toggle HUD (показать/скрыть)
F2	Cycle placement material (Air/Glass/Fluid/FloorWhite/FloorGray)
F3	Reset camera (вернуть в начальную позицию)
F4	Toggle control mode (walk/creative/spectator)
F5	Cycle scene preset (VoxelLab / FlatBenchmark / TransparencyStress / ChunkGrid / MeshingStress)
F6	Save world snapshot (в файл)
F7	Load world snapshot (из файла)
F8	Cycle editor tool
F9	Toggle chunk bounds (визуализация границ)
F10	Toggle dirty-chunk overlay
F11	Toggle walk air-control mode
F12	Toggle walk auto-jump delay
G	Toggle detailed HUD (доп. метрики GPU)
P	Toggle pause
C	Capture screenshot (сохранить в PNG)
H / K	Decrease / increase lighting exposure
N	Cycle tone-map operator
J	Toggle walk auto-jump
V	V-sync toggle (IMMEDIATE → MAILBOX → FIFO)
Esc	Выход

Defence r0 hotkeys (см. `src/app/main.cpp:519-540`, обходят формальный `InputAction` enum):

Клавиша	Действие
F5	(override) Hot-reload шейдеров через <code>RebuildAllShadersFromDisk()</code>
F6	(override) Toggle ray-march pass
V	(override) V-sync toggle (см. выше)

2.3. Типовые сценарии (How-to)

2.3.1. Как запустить сцену VoxelLab?

1. Запустите ProjectV (см. §Инструкция по развёртыванию).
2. По умолчанию откроется сцена VoxelLab (`PROJECTV_SCENE_PRESET=VoxelLab`).
3. Используйте WASD + мышь для перемещения по сцене.

2.3.2. Как разместить воксель?

1. Наведите камеру на пустую ячейку (Air).
2. Нажмите **ПКМ** (правая кнопка мыши).
3. Разместится воксель с материалом по умолчанию (FloorWhite).
4. Чтобы изменить материал, нажмите F2 для cycle по материалам (Air → Glass → Fluid → FloorWhite → FloorGray).

2.3.3. Как удалить воксель?

1. Наведите камеру на solid-воксель.
2. Нажмите **ЛКМ** (левая кнопка мыши).
3. Воксель заменится на Air.

2.3.4. Как перезагрузить шейдеры без перезапуска?

1. Нажмите F5 (defence r0 hotkey — обходит InputAction::CycleScenePreset).
2. Все шейдеры будут перекомпилированы из src/shaders/*.comp/*.vert/*.frag через glslc и перезагружены в GPU pipelines.

2.3.5. Как переключить пресет сцены?

- В runtime: нажмите F5 (InputAction::CycleScenePreset). Цикл: VoxelLab → FlatBenchmark → TransparencyStress → ChunkGrid → MeshingStress → VoxelLab.
- Через env: установите PROJECTV_SCENE_PRESET=MeshingStress перед запуском.

2.3.6. Как снять capture для профилирования?

1. Включите Debug HUD (G).
2. Нажмите C (InputAction::CaptureScreenshot) — сохранит PNG.
3. Или запустите ./tools/linux/Invoke-ProjectVRuntimeSmoke.sh (или tools/windows/Invoke-ProjectVRuntimeSmoke.ps1 на Windows) — автоматически сгенерирует 6 captures.

2.3.7. Как включить Vulkan validation layers?

- При debug-сборке: установите PROJECTV_ENABLE_VALIDATION=ON (по умолчанию включено).
- Вручную: export VK_INSTANCE_LAYERS=VK_LAYER_KHRONOS_validation перед запуском.

2.4. FAQ

Q1. Приложение запускается, но окно чёрное / не рендерит. А: Проверьте, что Vulkan validation layers не выдают ошибок. Запустите с PROJECTV_ENABLE_VALIDATION=ON, откройте консоль (нет консоли — смотрите stderr), проверьте ошибки. Частая причина — устаревший драйвер GPU.

Q2. FPS ниже 60 на VoxelLab. А: Проверьте, что GPU поддерживает Vulkan 1.4. Включите Debug HUD (G) — если GPU time > 16 ms, проблема в рендере; если CPU time > 16 ms — в физике/asset pipeline. Попробуйте пресет ChunkGrid для baseline.

Q3. После изменения шейдера в src/shaders/ ничего не происходит. А: F5 (defence r0 hotkey) перезагружает шейдеры в runtime. Убедитесь, что файл шейдера не содержит синтаксических ошибок (проверьте через glslc напрямую). После F5 в stderr должно появиться [shader reload] ... сообщение.

Q5. VoxelLab tremor при включённом TAA (BUG-004). А: Известная проблема — descriptor race в TAA pass. Меши слегка дрожат на статичной сцене. Обходной путь: отключить TAA через PROJECTV_RENDERER_TAA=OFF. Bug отслеживается в docs/KT-3.2_Final_Report.md (post-mortem 3).

Q6. ctest выдаёт 11/12 passing, какой suite упал? А: Запустите ctest --output-on-failure. Verbose-вывод покажет, какой тест и где упал. На 2026-06-13 все 12 suites passing.

Q8. Где логи runtime? А: В stderr (нет UI console) + Tracy capture (если включён через PROJECTV_TRACY=ON). Также runtime/scene.json хранит последний пресет сцены.

Q9. Как добавить новый пресет сцены? А: Все пресеты определены в `src/voxel/SceneConfig.cpp` в `ParseScenePreset()`. Чтобы добавить новый: добавьте `if (text == "MyNewPreset") { ... return true; }` в эту функцию и соответствующий `enum` в `core/Types.hpp`. Пересоберите проект.

Q10. Где исходный код алгоритма X? А: Все ключевые алгоритмы (`game loop`, `greedy-мешинг`, `walk authority`, `scene-fitted shadow`, `fluid CA`) перечислены в `docs/КТ-2.1_Architecture.md` в разделе «Описание ключевых алгоритмов» с указанием модуля и шейдера, где каждый применяется. Для поиска конкретного файла реализации используйте `rg` по имени алгоритма в коде.